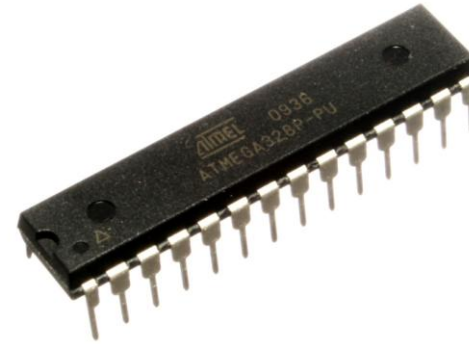
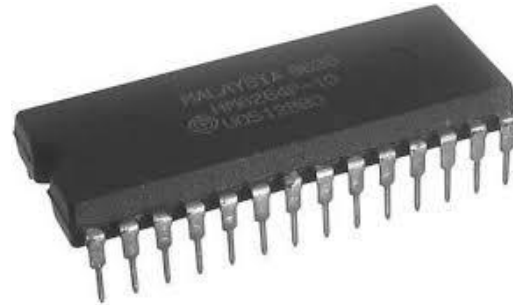
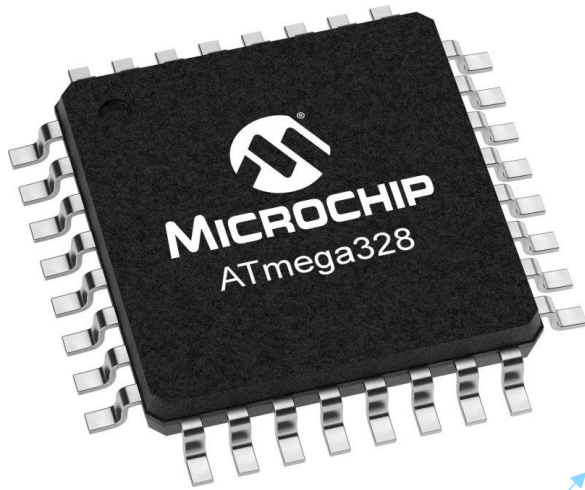


MONITORAMENTO AMBIENTAL COM SENSORES E MICROCONTROLADORES

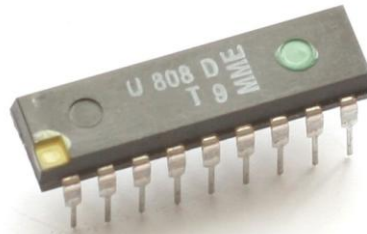
AULA 2

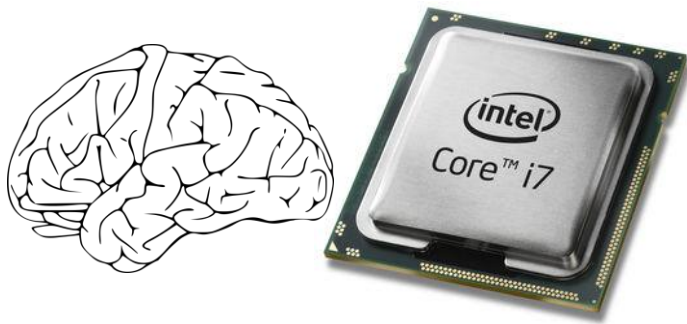
Fundamentos de sistemas embarcados





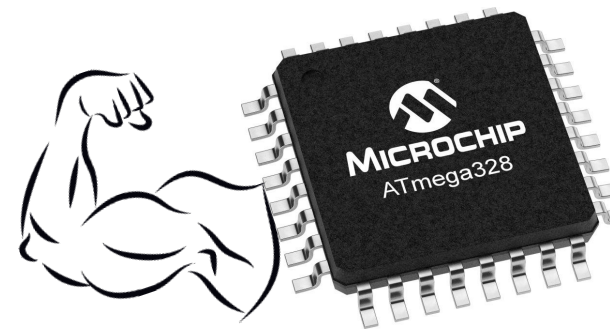
Microcontroladores & Microprocessadores (CPU)





Microprocessador

Periféricos (Memória)	Dependente de periféricos
Aplicações	Computadores e servidores
Custo	Mais caro
RAM	Sem restrição pois utiliza memória externa ($10^6 - 10^9$ B)
Consumo de energia	Consome mais energia e dissipa mais calor
Clock	$\mathcal{O}$ (GHz)



Microcontrolador

Sistema integrado
Dispositivos embarcados
Mais barato
Possui pouca memória (10^3 B)
Consome pouca energia
$\mathcal{O}$ (MHz)

Dispositivos utilizados para facilitar a prototipagem:

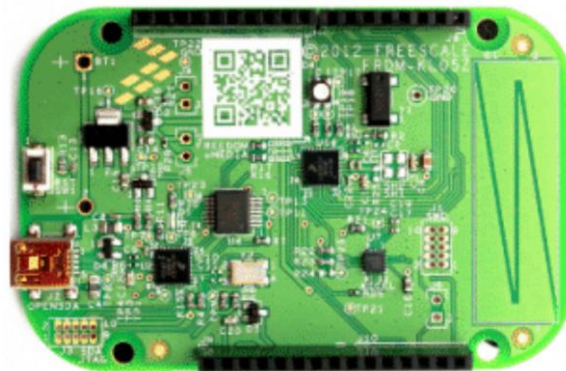
base de um microprocessador
que possui um software dedicado
ao sistema que controla

Microcontroladores &
Microprocessadores



Placa
Arduino

microcontrolador



microprocessador



Placa
Raspberry Pi



Arduino UNO

Arduino Nano



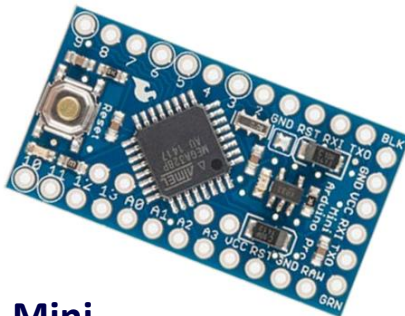
Arduino
Leonardo



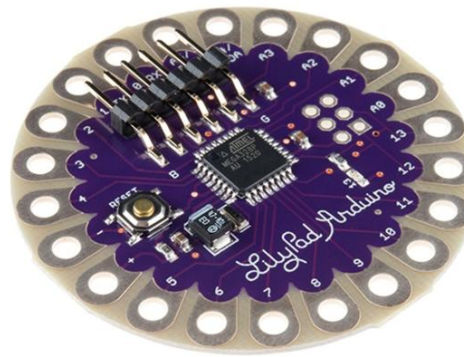
Um mesmo objetivos para diferentes aplicações !

Diferentes placas Arduino

Variante: microcontrolador, número de portas, memória, clock, conexão, tensão de operação, formas e tamanhos



Arduino Mini



Arduino
LilyPad



Arduino
MEGA

Arduino



A blue prohibition sign (a circle with a diagonal line) over the text "Cartão SD".

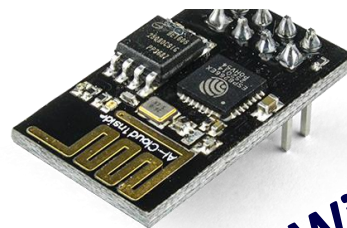
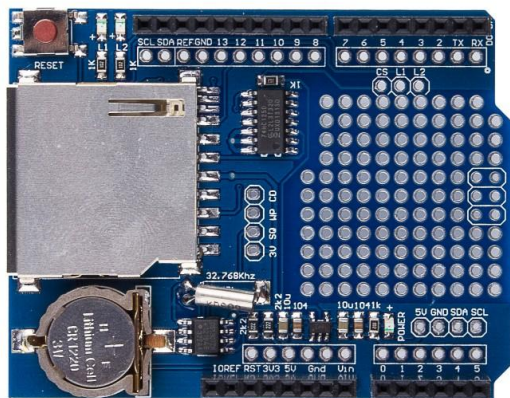


Mais portas





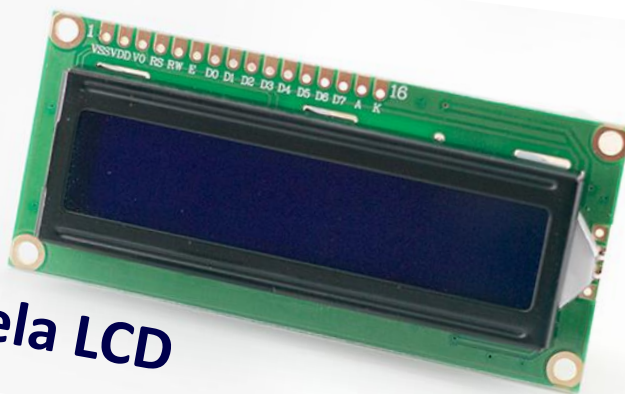
Cartão SD



Wi-fi



Mais portas

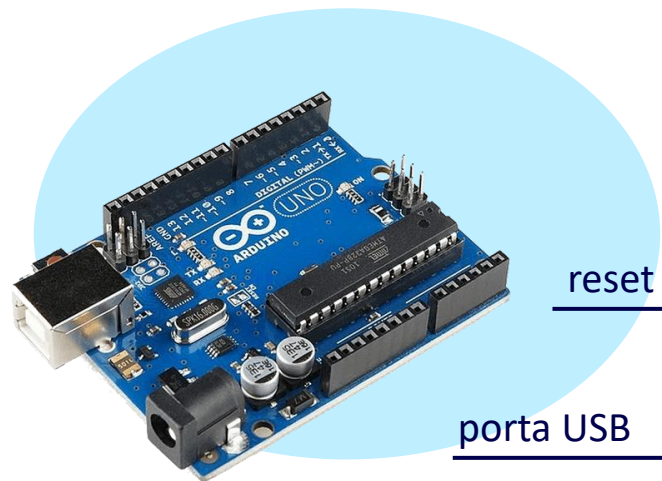


Tela LCD



Som

Shields



reset

porta USB

Tensão de operação: 6 – 20 V
recomendada: 7 – 12 V

alimentação externa

tensão de referência para shield

pino terra

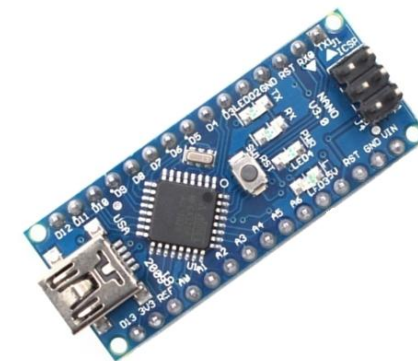
14 portas digitais (6 PWM)

Microcontrolador
Atmega328

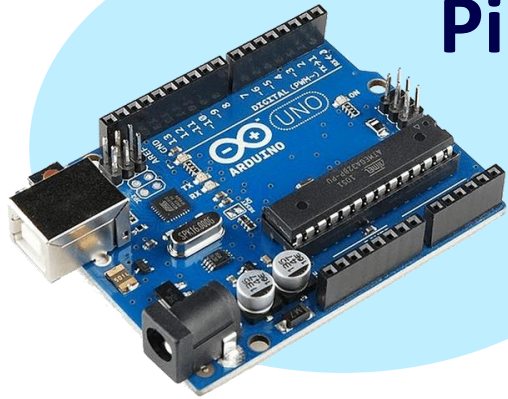
6 portas analógicas

energizar um módulo externo ou o próprio Arduino
(mesma tensão de entrada (externa) se utilizada para energizar modulo)

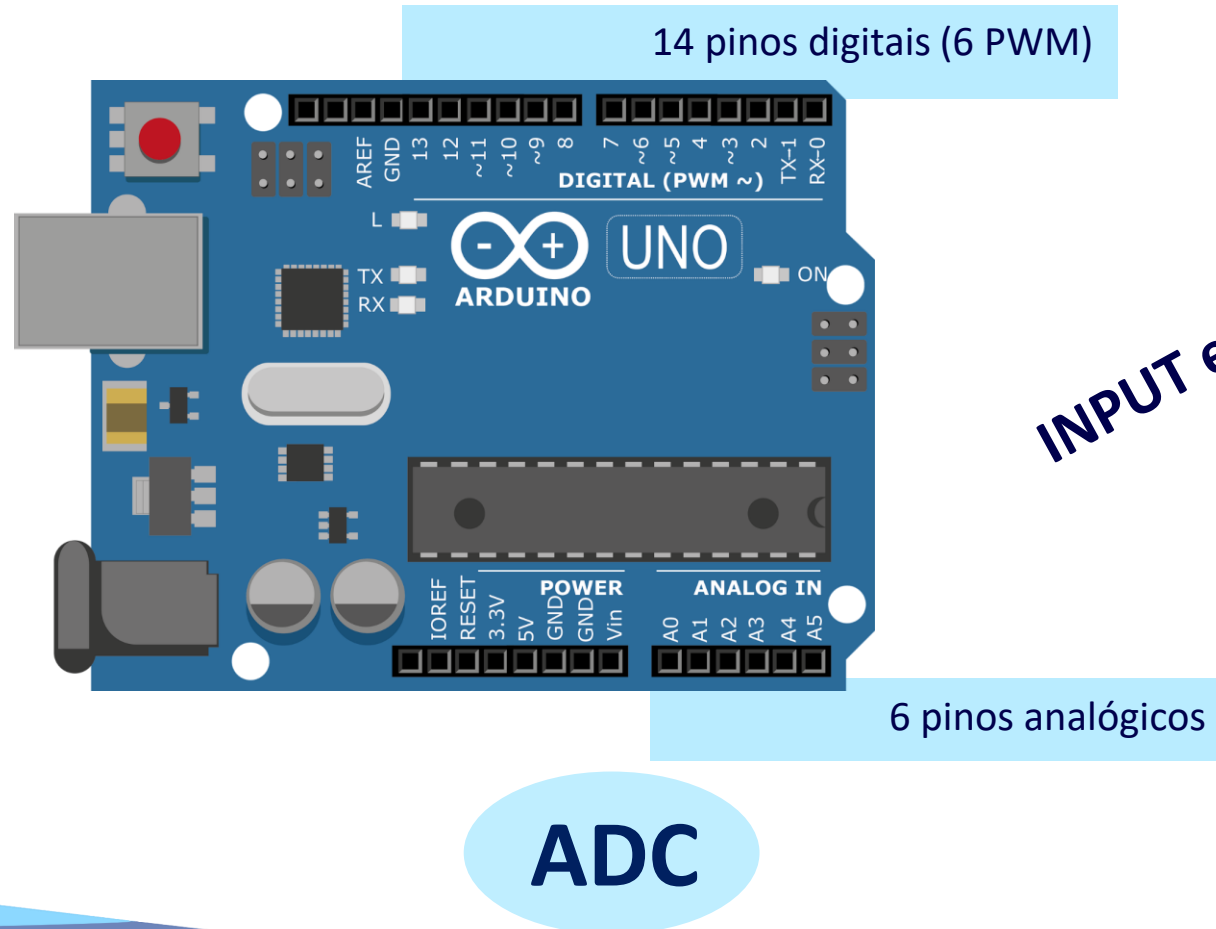
fornece 5V para o circuito externo



2.2 – Portas analógicas e digitais



Pinos digitais & analógicos

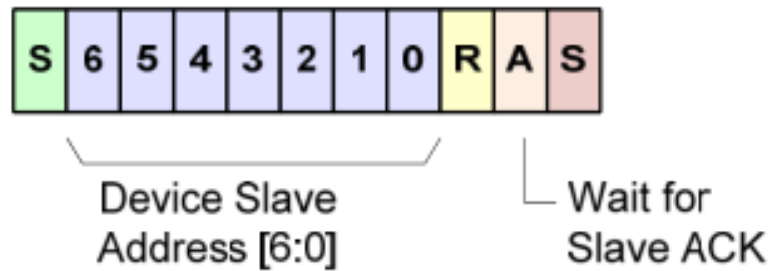


Porta digital

1 bit = $2^1 = [1, 0]$

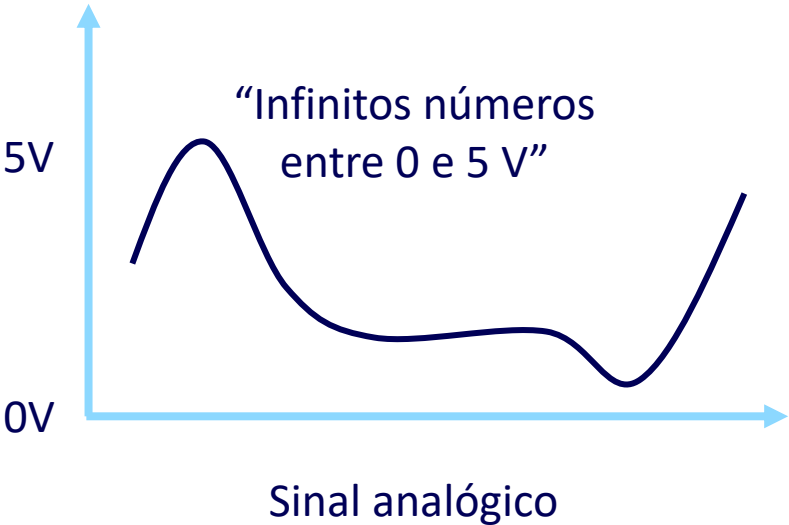
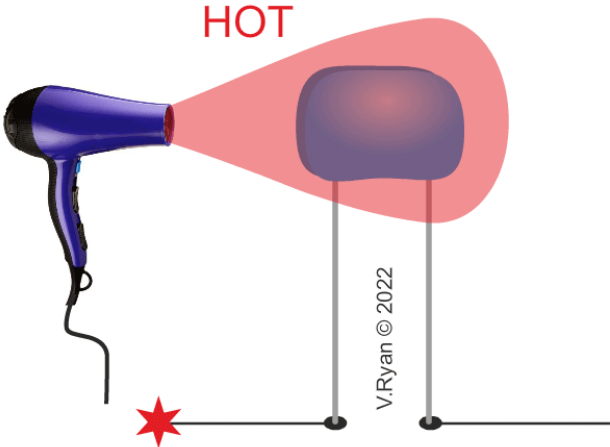
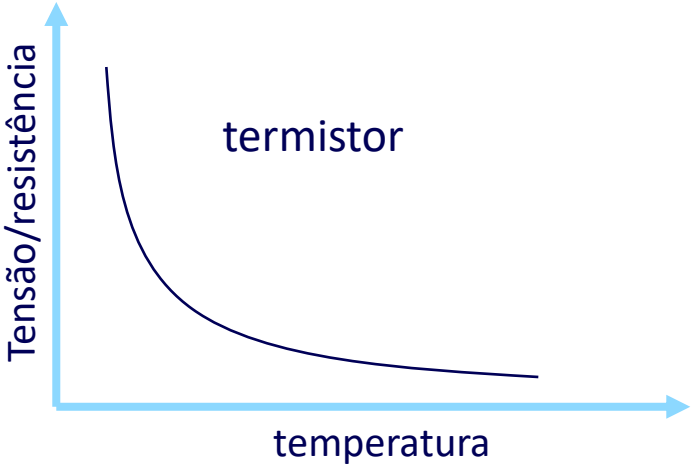
1 bit = $2^1 = [\text{HIGH}, \text{LOW}]$

1 bit = $2^1 = [5 \text{ V}, 0 \text{ V}]$



Decimal	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

Conversor analógico-digital (ADC)



Decimal to Binary Conversion

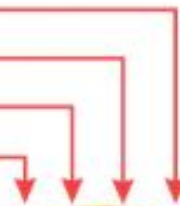
Step 1: Divide the given number **13** repeatedly by 2 until you get "0" as the quotient

$$13 \div 2 = 6 \text{ (Remainder 1)}$$

$$6 \div 2 = 3 \text{ (Remainder 0)}$$

$$3 \div 2 = 1 \text{ (Remainder 1)}$$

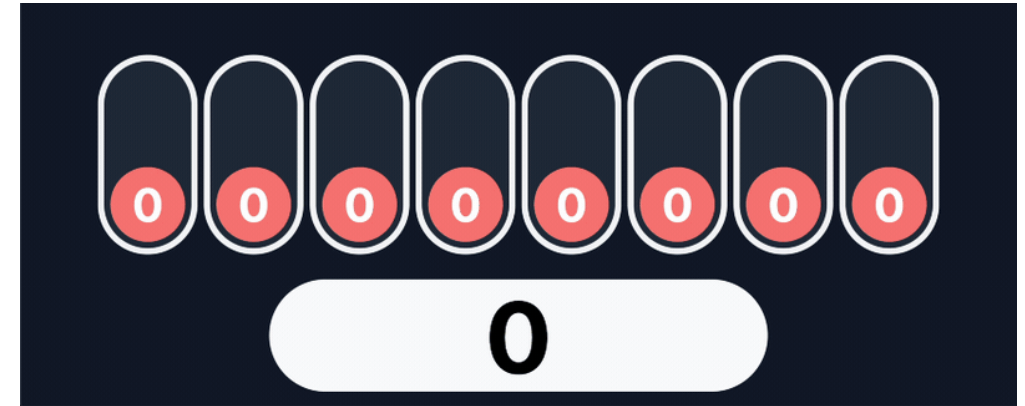
$$1 \div 2 = 0 \text{ (Remainder 1)}$$



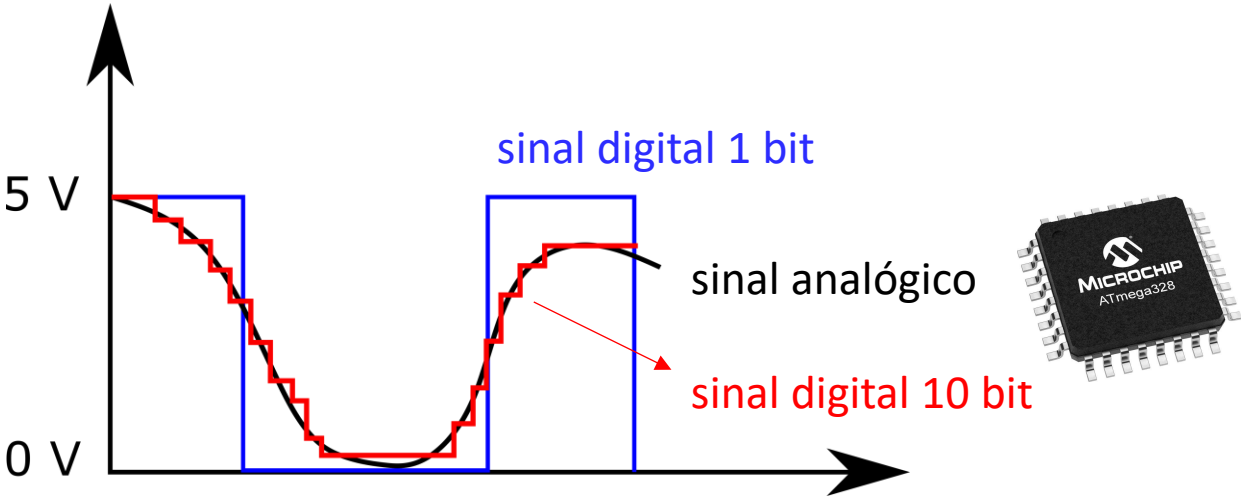
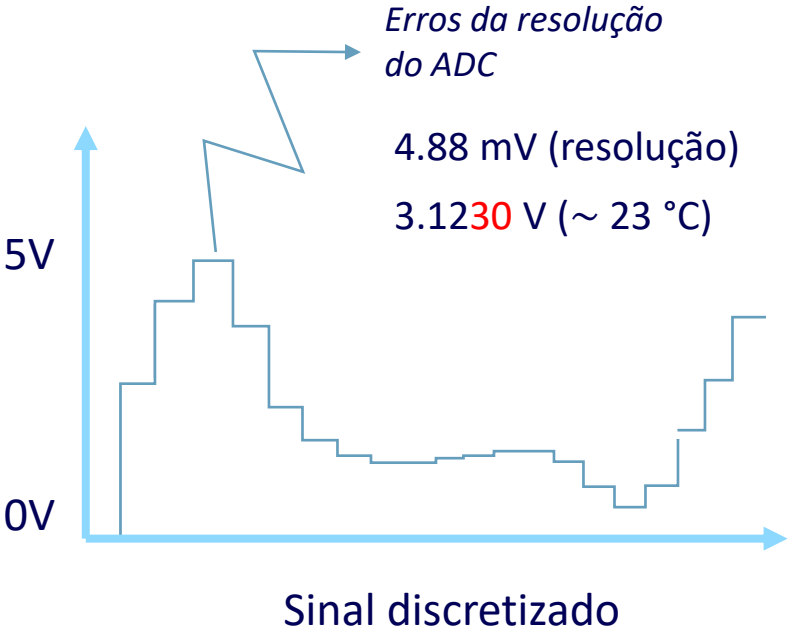
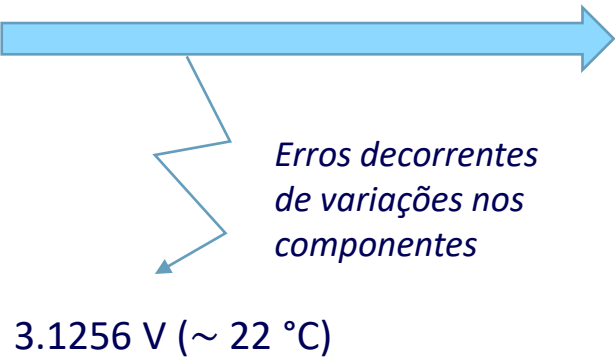
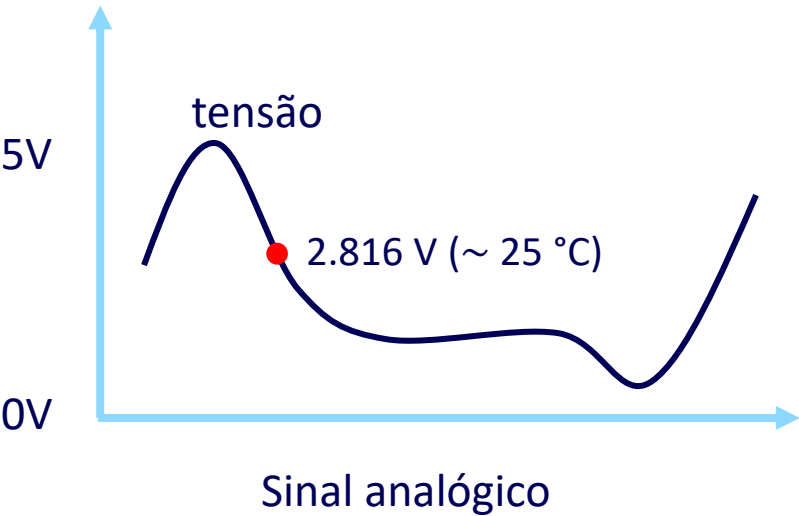
Step 2: Write the remainders in the revers **1 1 0 1** order

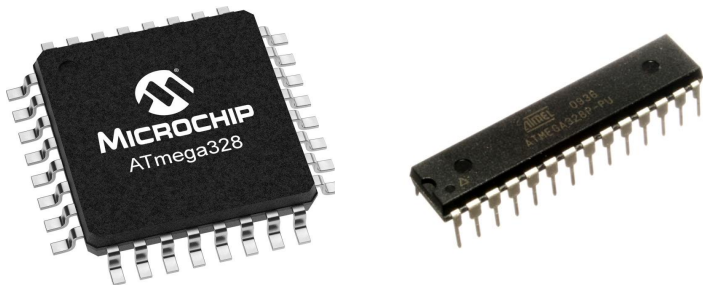
$$\therefore 13_{10} = 1101_2$$

(Decimal) (Binary)



Conversor analógico-digital (ADC)



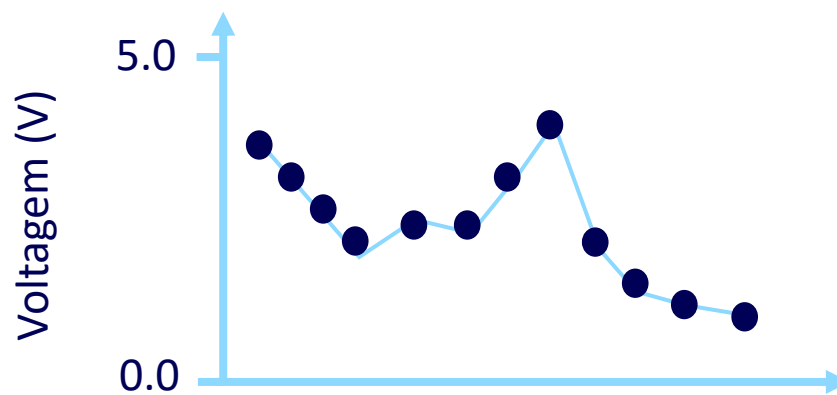


ATmega328 | Arduino

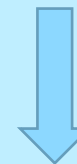
**Sinal analógico
entrando**



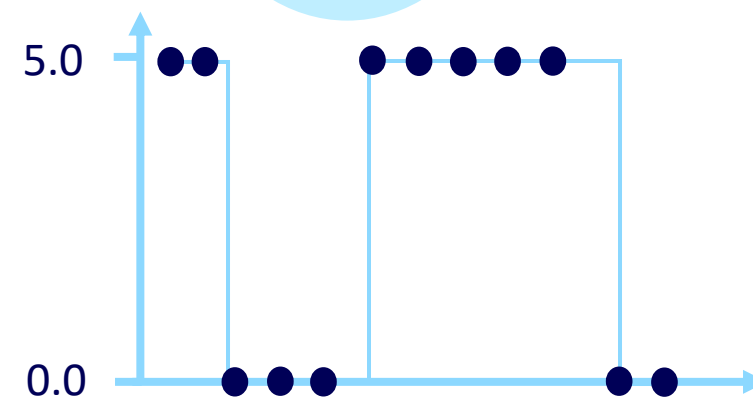
Possuí um ADC de 10
bits para sinal analógico
de 0 V a 5 V



**Sinal digital
entrando**



Sinal digital binário
(1 bit)



Leitura na
porta analógica

10 Bits

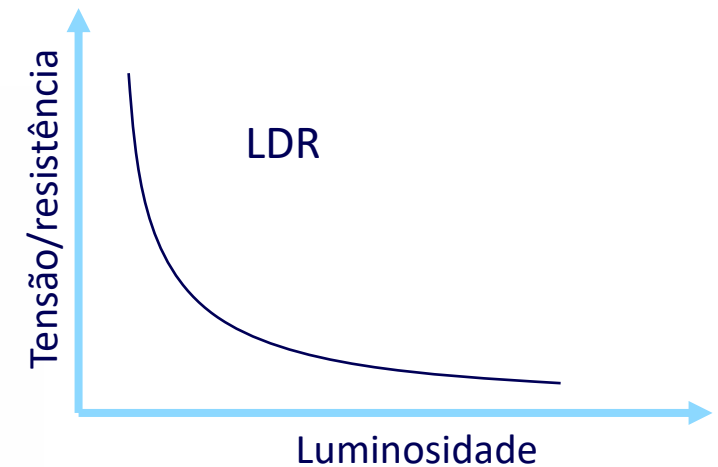
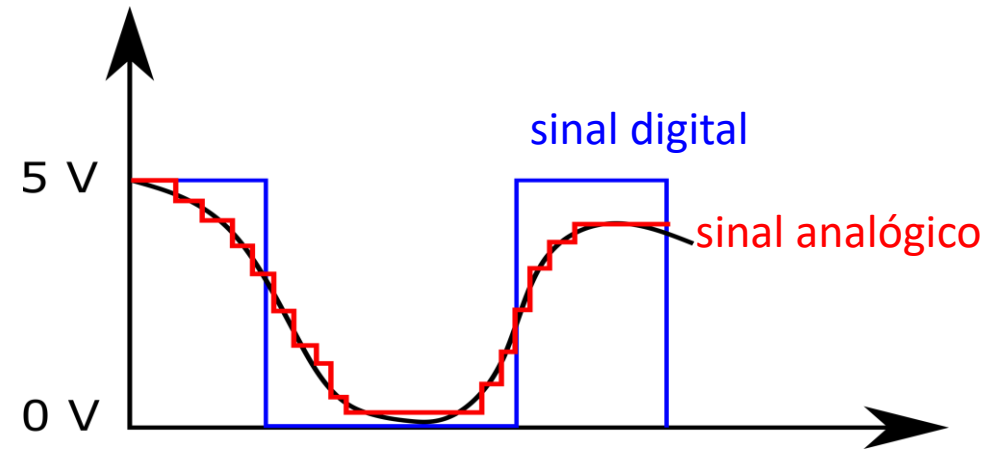
$$2^{10} = 1024$$

Leitura na
porta digital

HIGH or LOW
5V or 0V

$$1 \text{ Bit} = 2$$

Sensor analógico



Enviando
informações
pela porta
digital PWM

8 Bits

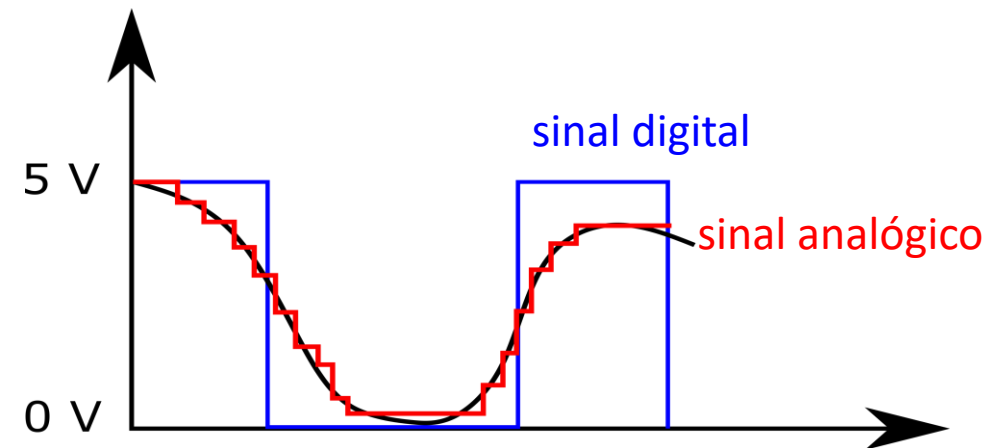
$$2^8 = 256$$



Enviando
informações
pela porta
digital

HIGH or LOW
5 V or 0 V

$$1 \text{ Bit} = 2$$



Leitura na
porta analógica

10 Bits

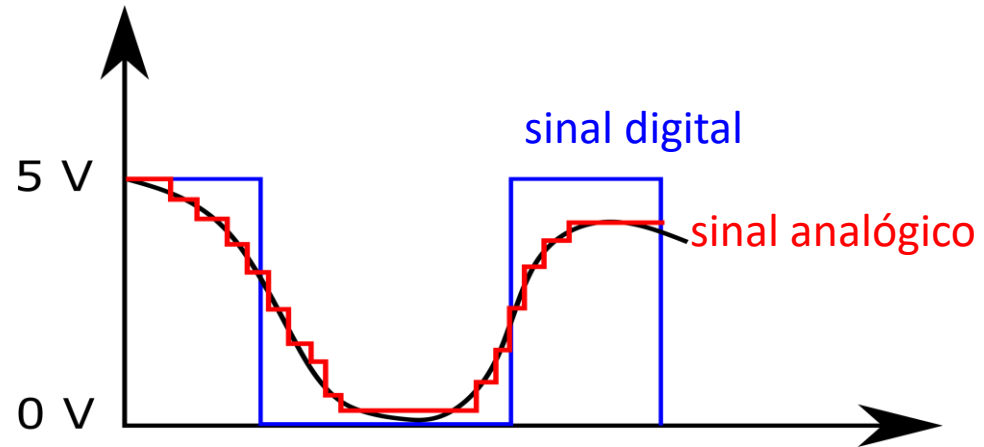
$$2^{10} = 1024$$

Leitura na
porta digital

HIGH or LOW

5 V or 0 V

$$1 \text{ Bit} = 2$$



Inicialização o IDE do Arduino e a estrutura do código

Compila o código e carrega para a placa Arduino

Salva o sketch

Compila o código

Cria um novo sketch

Apresenta um menu de todos os sketches já existentes

void loop()

Faz exatamente o que a função sugere: entra em um looping infinito.

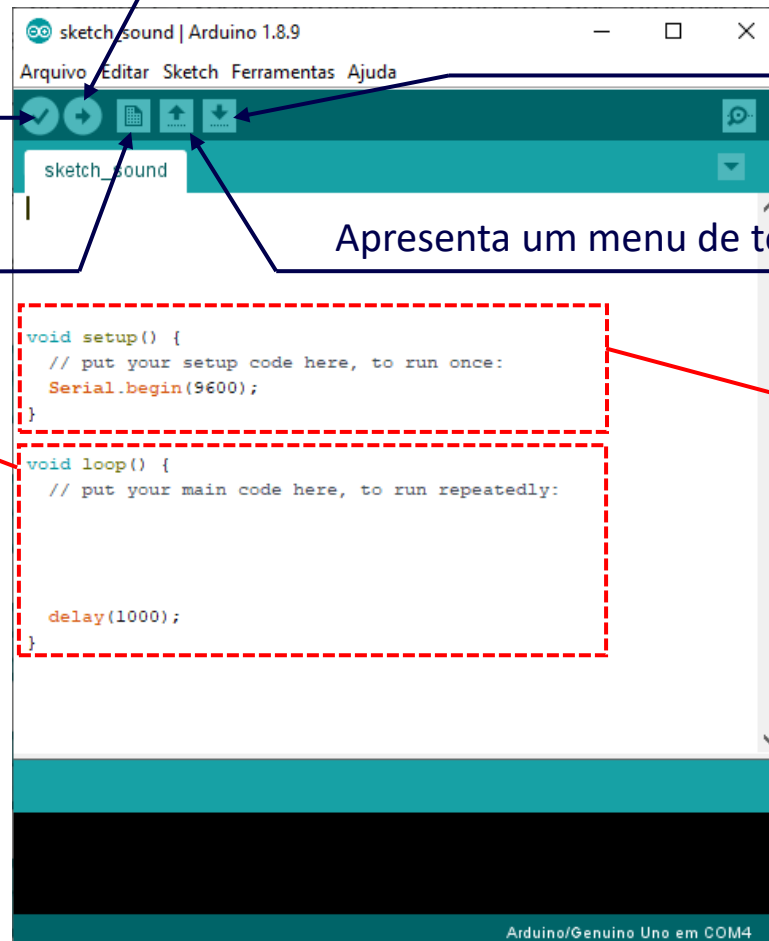
Responsável por controlar ativamente a placa, recebendo e enviando comandos

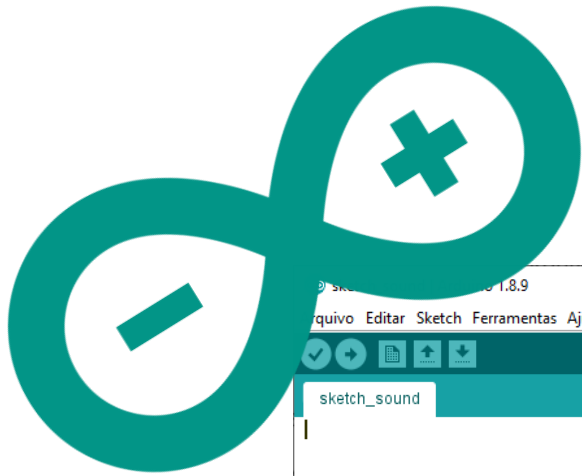
*Função **delay()** informa ao Arduino para esperar durante 1000 milissegundos (1 segundo) para as próximas instruções*

void setup()

Chamada quando o programa começa a rodar (uma única vez)

Função para declarar variáveis, os modos dos pinos utilizados, informações sobre o programa para os usuários





```
Arduino IDE 1.8.9
Arquivo Editar Sketch Ferramentas Ajuda

sketch_sound

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
}

void loop() {
  // put your main code here, to run repeatedly:

  delay(1000);
}

Arduino/Genuino Uno em COM4
```

Variáveis:

Tipos

Inteiros

Float

Char

Booleanas

Exemplos

```
int i = 13;
```

```
float y = 2.75;
```

```
char let = 'A';
```

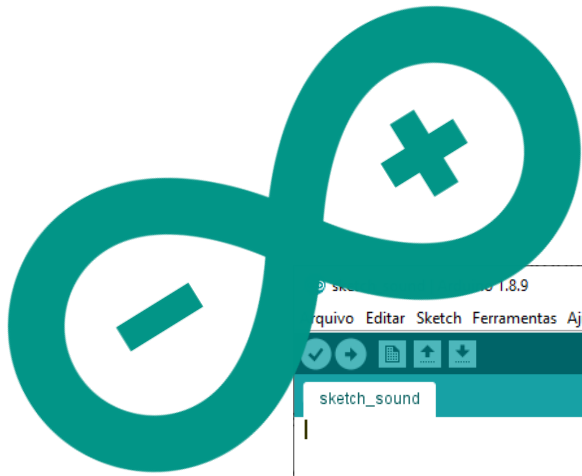
```
boolean io = false;
```

Estruturas de controle:

```
if (condição 1) {
    comando 1
}
elseif (condição 2){
    comando 2
}
else {
    comando 3
}
```

Alternativa:

```
switch(variável) {
    case valor1:
        comando 1
        break;
    case valor2:
        comando 2
        break;
    default:
        comando 3
        break;
}
```



```
Arquiteto 1.8.9
Arquivo Editar Sketch Ferramentas Ajuda

sketch_sound

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
}

void loop() {
  // put your main code here, to run repeatedly:

  delay(1000);
}

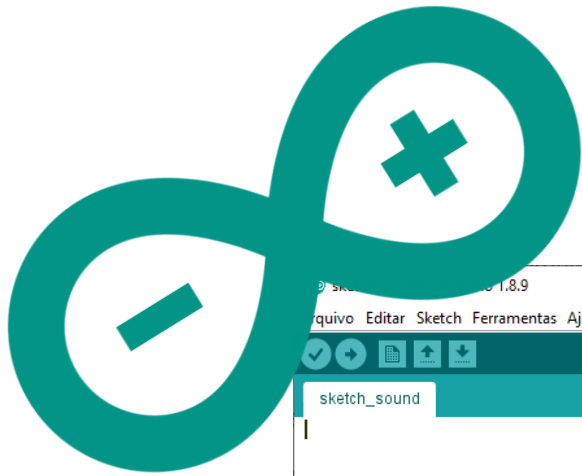
Arduíno/Genuíno Uno em COM4
```

Estruturas de repetição:

```
while (condição) {  
  comando  
}
```

```
for (inicialização; condição; incremento){  
  comando  
}
```

```
do {  
  comando  
} while(condição);
```



```
Arduino IDE 1.8.9
Arquivo Editar Sketch Ferramentas Ajuda

sketch_sound

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
}

void loop() {
  // put your main code here, to run repeatedly:

  delay(1000);
}

Arduino/Genuino Uno em COM4
```

Operadores de comparação e booleanos:

Operadores

Significado

`!=`

Diferente de

`<`

Menor que

`>=`

Maior que ou igual a

`==`

Igual a

`>`

Maior que

`>=`

Maior que ou igual a

`!=`

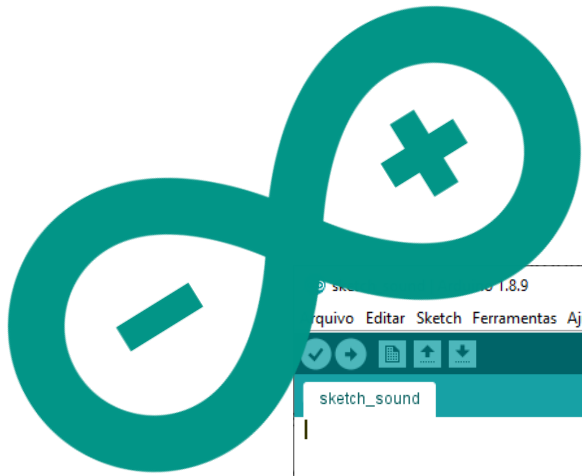
NÃO lógico

`&&`

E lógico

`||`

OU lógico



```
Arduino IDE 1.8.9
Arquivo Editar Sketch Ferramentas Ajuda

sketch_sound

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
}

void loop() {
  // put your main code here, to run repeatedly:

  delay(1000);
}

Arduino/Genuino Uno em COM4
```

Operadores Aritméticos:

Operadores

Significado

%

Resto de divisão inteira

*

Multiplicação

+

Adição

-

Subtração

/

Divisão

=

Operador de atribuição

!

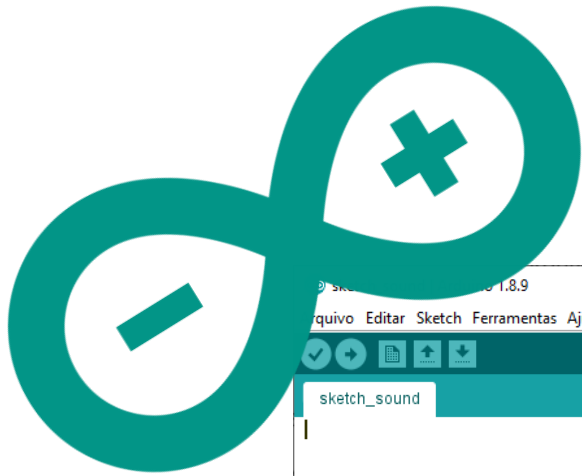
NÃO lógico

++

Incremento

--

Decremento



```
Arquiteto 1.8.9
Arquivo Editar Sketch Ferramentas Ajuda

sketch_sound

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
}

void loop() {
  // put your main code here, to run repeatedly:

  delay(1000);
}

Arduino/Genuino Uno em COM4
```

Outras funções e estruturas interessantes:

return x;

Exatamente como utilizada em C++ para definir e enviar valores de funções

Sair de laços for, while e do... while, ignorando a condição do loop

break;

continue;

O comando “pula” o resto da iteração atual de um loop, mas verifica a condição subsequente

A **GRANDE** diferença

pinMode()
Exigência para pinos digitais

Configura o pino específico como entrada (input) ou saída (output)

```
pinMode (8, OUTPUT);
```

digitalWrite()

Escreve um valor (HIGH ou LOW) de um pino digital específico

```
digitalWrite (8, HIGH);
```

digitalRead()

Lê o valor (HIGH ou LOW) de um pino digital específico

```
x = digitalRead (8);
```

analogWrite()

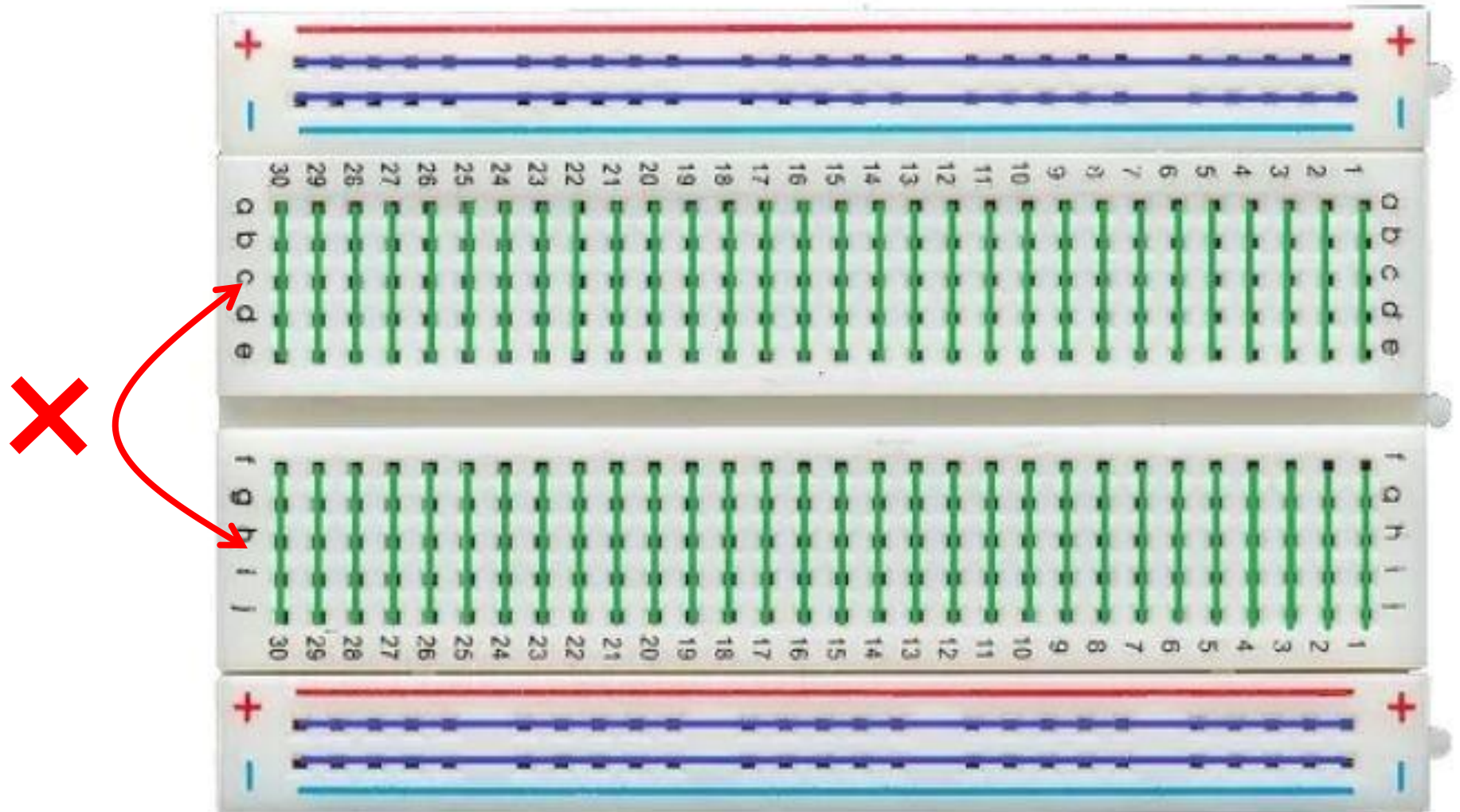
Escreve um valor analógico (onda PWM) em um pino digital PWM específico

```
analogWrite (9, valor qualquer);
```

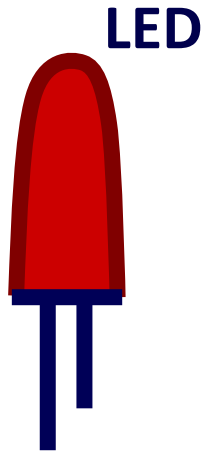
analogRead()

Lê o valor de um pino analógico específico

```
int x = analogRead (A0);
```



2.3 – Programando um microcontrolador

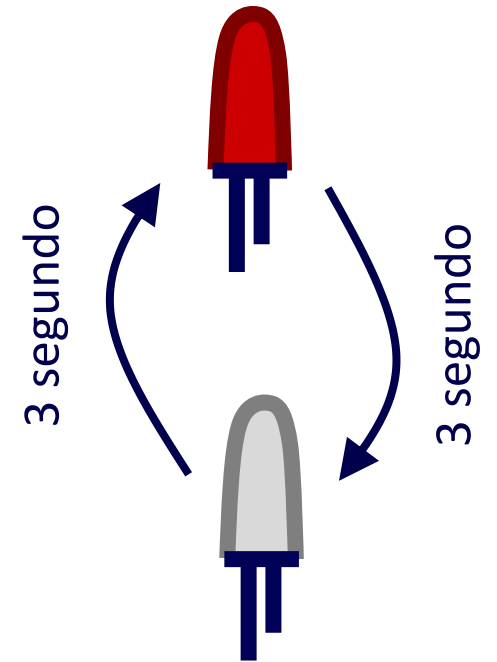


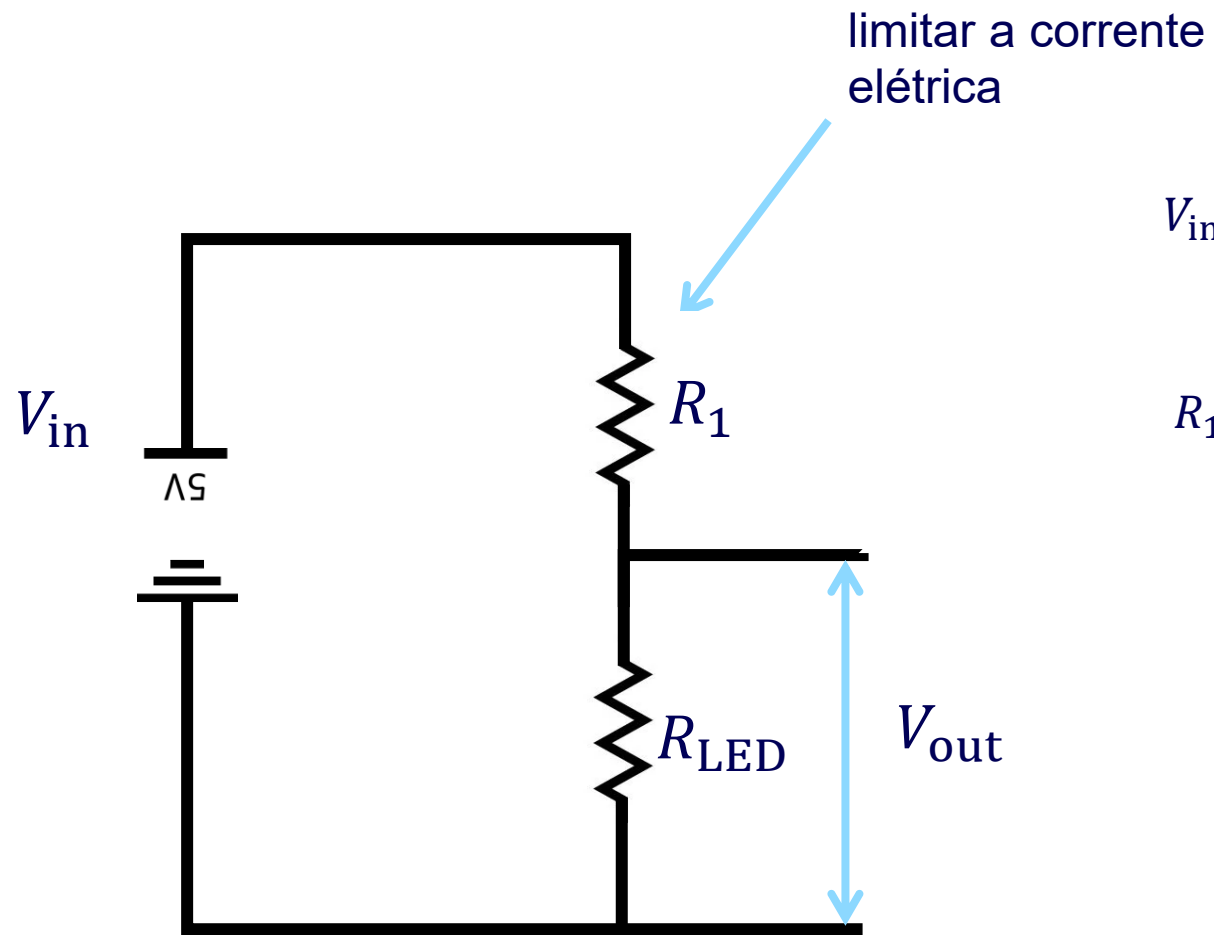
Nós devemos dar um comando ao Arduino para controlar o LED !!!

Exemplo:
Queremos que o LED acenda e apague em intervalos

Programa:

```
LED    → ON  
delay  → 3 seg  
LED    → OFF  
delay  → 3 seg
```

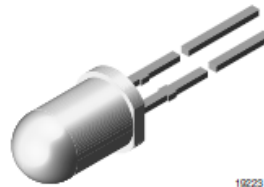




$$V_{in} = V_{R1} + V_{LED} \rightarrow V_{R1} = V_{in} - V_{LED}$$

$$R_1 = \frac{V_{in} - V_{LED}}{i} = \frac{V_{in} - V_{LED}}{i_{LED}}$$

Ultrabright White LED, Ø 5 mm Untinted Non-Diffused Package



19223

DESCRIPTION

The VLHW5100 is a clear, non-diffused 5 mm LED for high end applications where supreme luminous intensity required.

These lamps with clear untinted plastic case utilize the highly developed ultrabright InGaN technologies.

The lens and the viewing angle is optimized to achieve best performance of light output and visibility.

PRODUCT GROUP AND PACKAGE DATA

- Product group: LED
- Package: 5 mm
- Product series: standard
- Angle of half intensity: $\pm 10^\circ$

FEATURES

- Untinted non-diffused lens
- Utilizing ultrabright InGaN technology
- High luminous intensity
- Luminous intensity and color categorized for each packing unit
- ESD-withstand voltage: up to 4 kV according to JESD22-A114-B
- Circuit protection by Zener diode
- Material categorization: for definitions of compliance please see www.vishay.com/doc?99912



RoHS
COMPLIANT
HALOGEN
FREE
GREEN
Product

APPLICATIONS

- Interior and exterior lighting
- Outdoor LED panels
- Instrumentation and front panel indicators
- Replaces incandescent lamps
- Light guide compatible

FORWARD VOLTAGE (V)			at I_F (mA)
MIN.	TYP.	MAX.	
2.8	-	3.6	20

3.2 V

PARTS TABLE														
PART	COLOR	LUMINOUS INTENSITY (mcd)			at If (mA)	COORDINATE (x, y)			at If (mA)	FORWARD VOLTAGE (V)			TECHNOLOGY	
		MIN.	TYP.	MAX.		MIN.	TYP.	MAX.		MIN.	TYP.	MAX.		
VLHW5100	White	5600	-	11 200	20	-	0.33, 0.33	-	20	2.8	-	3.6	20	InGaN and converter

ABSOLUTE MAXIMUM RATINGS ($T_{amb} = 25^\circ\text{C}$, unless otherwise specified) VLHW5100				
PARAMETER	TEST CONDITION	SYMBOL	VALUE	UNIT
Reverse voltage		V_R	5	V
DC forward current		I_F	30	mA
Peak forward current	at 1 kHz, $t_p/T = 0.1$	I_{FSM}	0.1	A
Power dissipation		P_V	100	mW
Zener reverse current		I_Z	100	mA
Junction temperature		T_J	100	$^\circ\text{C}$
Operating temperature range		T_{amb}	-40 to +100	$^\circ\text{C}$
Storage temperature range		T_{stg}	-40 to +100	$^\circ\text{C}$
Soldering temperature	$t \leq 5$ s	T_{sd}	260	$^\circ\text{C}$
Thermal resistance junction-to-ambient		R_{thJA}	400	K/W

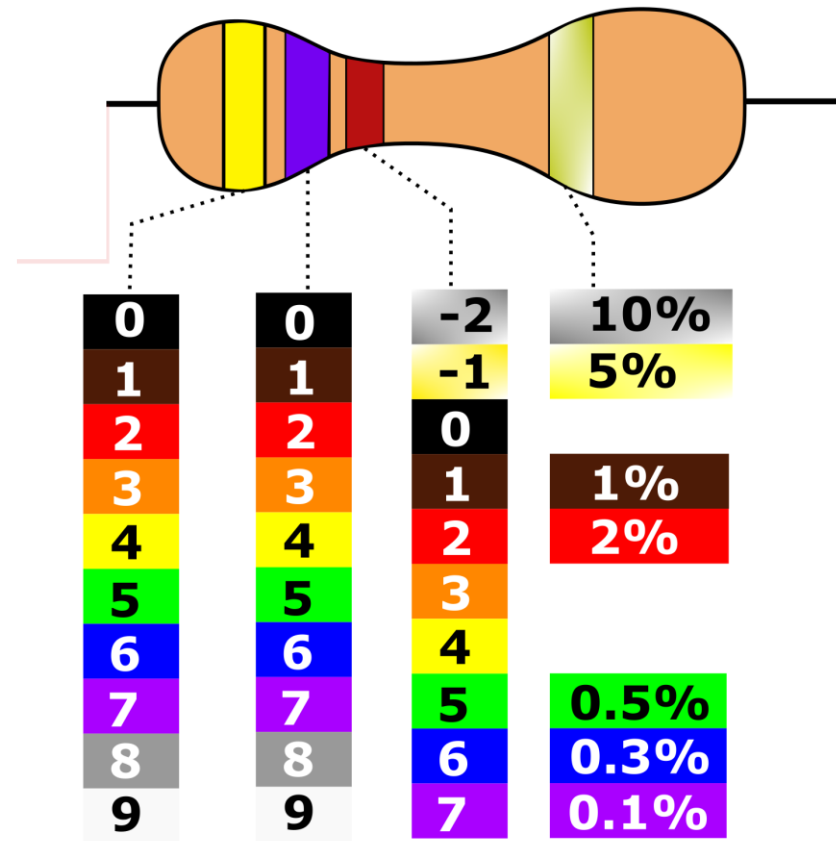
$$V_{\text{fonte}} = V_R + V_{\text{LED}}$$

$$V_R = 1,8 \text{ V}$$

Resistências disponíveis (código de cores)

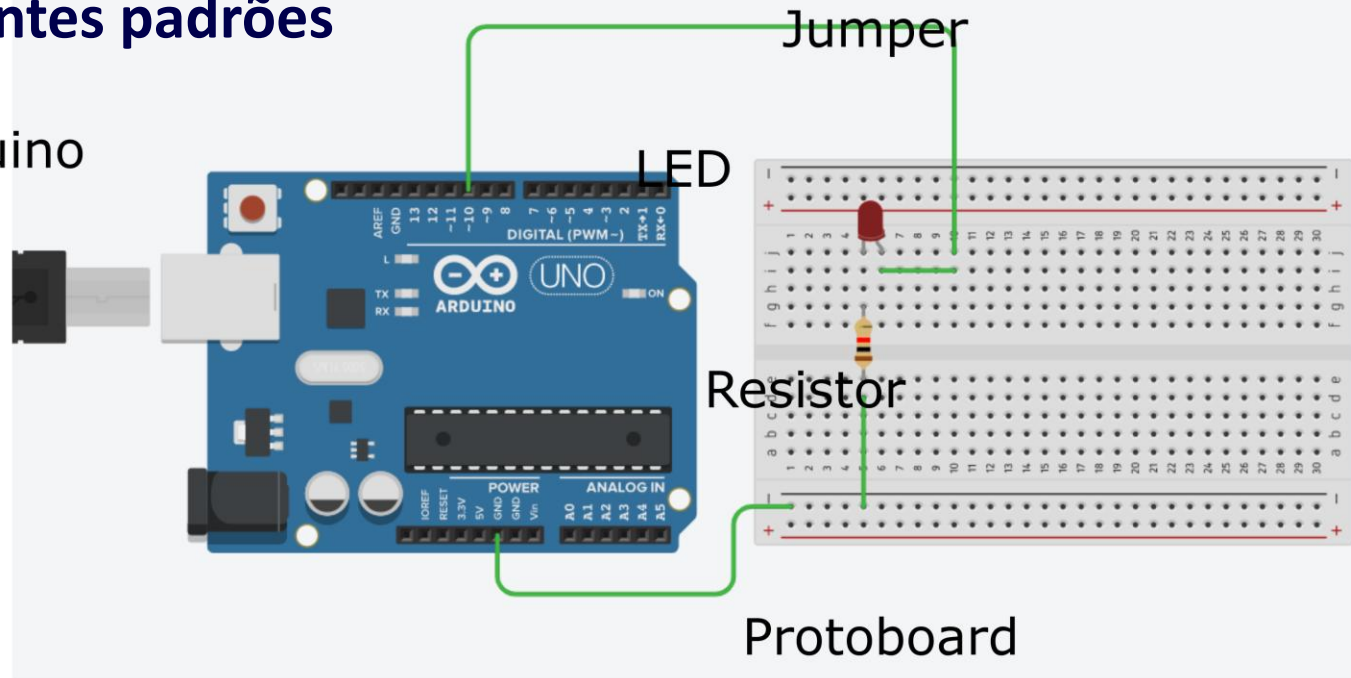
Não existem todas as combinações !!!

1. Optar pelo resistor com valor exato;
2. Criar um sistema em série e/ou paralelo para adaptar a resistência do circuito;
3. Optar pelo maior valor



Componentes padrões

Arduino



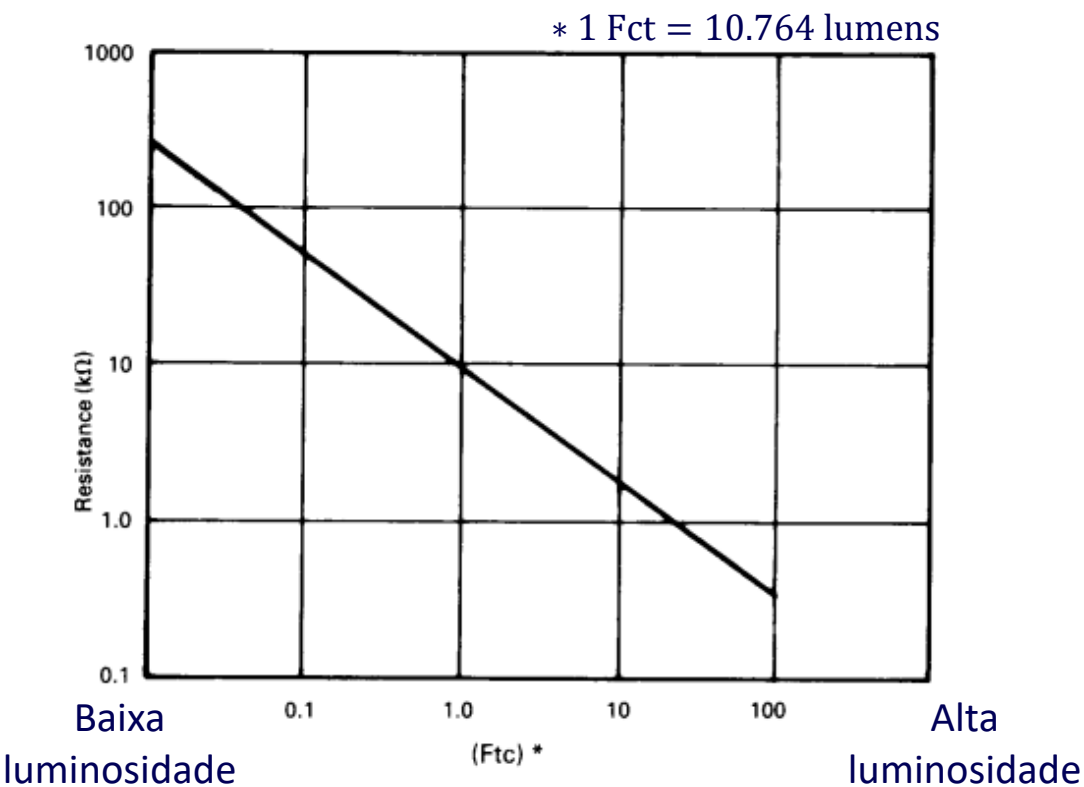
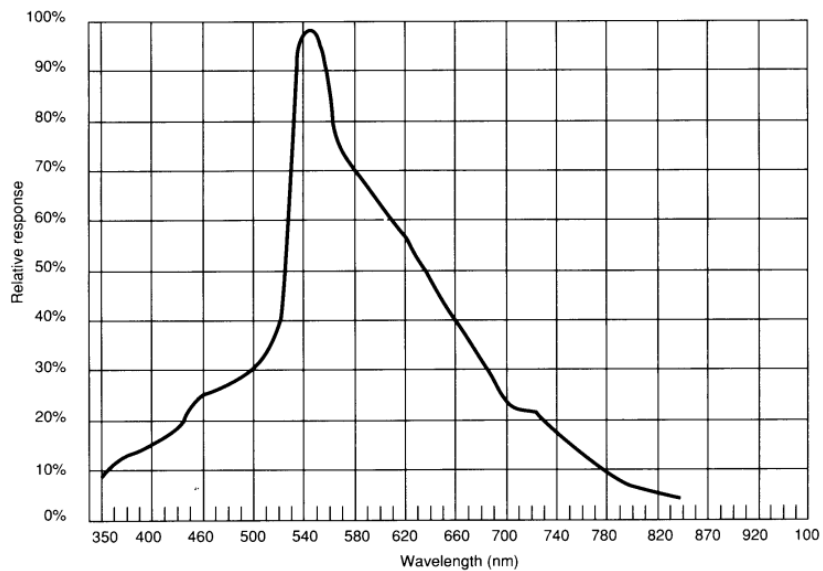
Caso você não tenha o Arduino
e uma protoboard com
componentes eletrônicos,
acesse o simulador online

Tinkercad



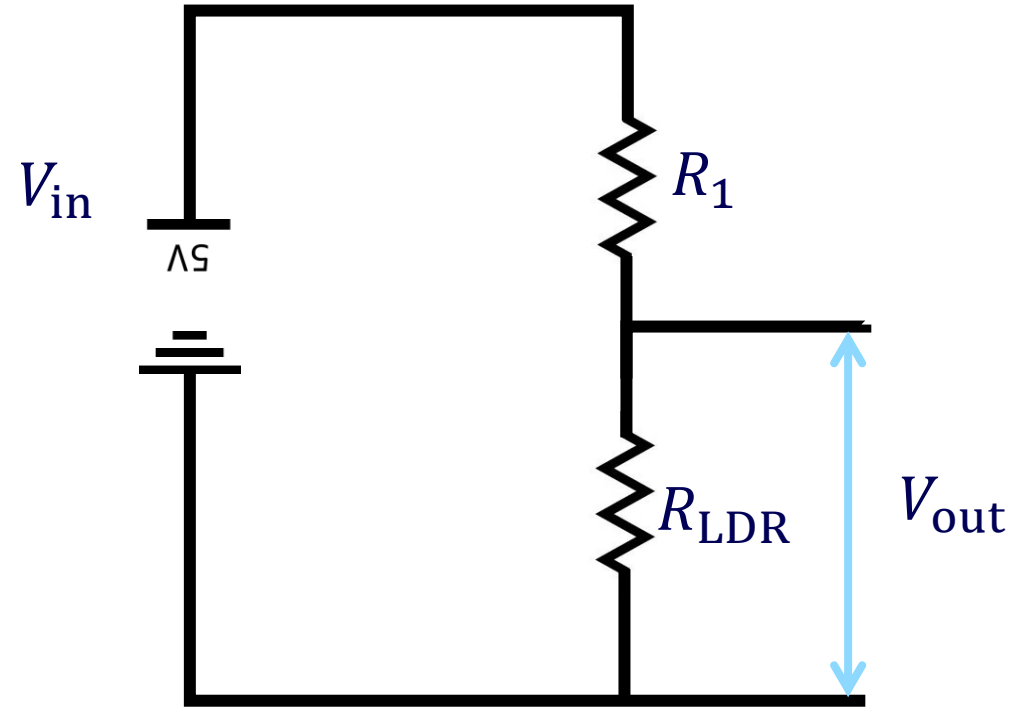
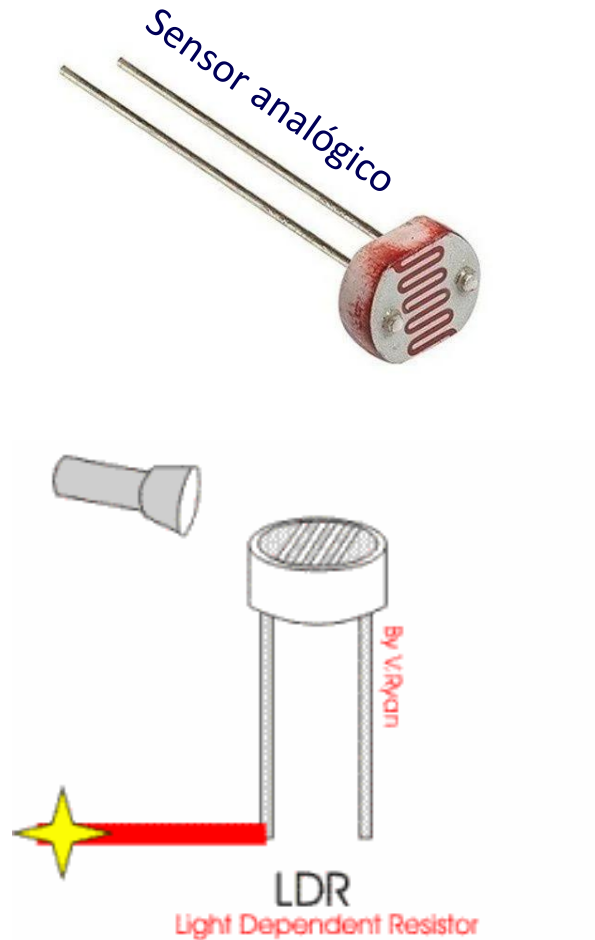
LDR (Light dependent resistor)

Sensor de luminosidade



LDR (Light dependent resistor)

Sensor de luminosidade



$$V_{in} = V_{R1} + V_{R2} \rightarrow i = \frac{V_{in}}{R_1 + R_{LDR}}$$

$$V_{out} = R_{LDR} i = V_{in} \left(\frac{R_{LDR}}{R_1 + R_{LDR}} \right)$$

Botão (Push button)

